On the Diversity of Coroutine Task Types

Document Number:	P4050R0
Date:	2026-03-15
Audience:	LEWG

Reply-to:	Vinnie Falco vinnie.falco@gmail.com

Table of Contents

Abstract

Revision History

- R0: March 2026 (post-Croydon mailing)

1. Disclosure

2. Dietmar Kühl

- 2.1 The Best Possible P3552R3

3. The Claim

4. `destructible`

5. Two Libraries, Two Environments

- 5.1 From trivial to standard divergence
- 5.2 Custom queries and `write_env`
- 5.3 What do real domains look like?
- 5.4 What about the standard queries?
- 5.5 `destructible`
- 5.6 If `Environment` has a default, who customizes it?

6. The Asio Precedent

7. Design Intent

8. The Ecosystem

9. Concepts Mitigate the Risk

10. Frequently Raised Concerns

11. Conclusion

Acknowledgments

References

- WG21 Papers

StackOverflow and GitHub

Talks

Other

Abstract

`std::execution::task<T, Environment>` (P3552R3[1]) is proposed as a lingua franca for coroutine-based asynchronous code. The `Environment` parameter is an open query-response protocol whose interoperability surface is defined by a single concept: `queryable`, which is `destructible`. This paper asks a simple question: when two libraries define different environments, how does one task `co_await` the other? The answer, traced step by step through the specification, is that no general conversion exists. The query set is open by design, and the only adaptation mechanism - `write_env` - requires the caller to know every missing query by name. The risk to the ecosystem is structural, documented by the specification itself, by NVIDIA's reference implementation, by the only production precedent (Boost.Asio), and by `task`'s own author.

This paper is one of a suite of six that examines the relationship between compound I/O results and the sender three-channel model. The companion papers are P4053R0[9], "Sender I/O: A Constructed Comparison"; P4054R0[10], "Two Error Models"; P4055R0[11], "Consuming Senders from Coroutine-Native Code"; P4056R0[12], "Producing Senders from Coroutine-Native Code"; and P4058R0[13], "The Case for Coroutines."

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version.

---

1. Disclosure

The author developed and maintains Corosio[34] and Capy[33] and believes coroutine-native I/O is the correct foundation for networking in C++. The cross-library bridges (Section 8) were authored by Klemens Morgenstern. The frame allocator gap was identified by Peter Dimov. Neither is a co-author. The author provides information, asks nothing, and serves at the pleasure of the chair.

The author regards `std::execution` as an important contribution to C++ and supports its standardization for the domains it serves well - GPU dispatch, heterogeneous execution, and compile-time work-graph composition among them. Nothing in this paper or its companions argues for removing, delaying, or diminishing `std::execution`. The author's position is narrower: that networking and stream I/O present a compound-result structure that the three-channel

model was not designed to carry, and that this domain is better served by a coroutine-native facility that can coexist with senders and interoperate where the domains meet. Two models, each correct for its domain, is a stronger standard than one model asked to serve both.

---

2. Dietmar Kühl

The findings documented in this paper and in P3801R0[6] are not engineering failures. They are structural consequences of making `task` serve two masters. Dietmar Kühl responded to every concern with professionalism and technical precision, and wrote P3796R1[7] to collect the issues himself.

The Environment parameter is not Dietmar's design choice. It is a structural requirement imposed by P2300.

2.1 The Best Possible P3552R3

Out of respect for Kühl's work, this paper evaluates only the strongest possible version of his design.

P3552R3[1] is underspecified in several areas. The `Environment` parameter lacks a default. The allocator delivery timing is being addressed in P3980R0[8]. The symmetric transfer gap is acknowledged in P3796R1[7] and P3801R0[6]. The author has contributed fixes to some of these gaps directly.

This paper grants P3552R3[1] every benefit of the doubt. We assume a default `Environment` will be provided. We assume the engineering gaps will be closed. We do not argue from underspecification. We evaluate the design on its strongest possible terms.

Three topics are off-limits. Allocator timing - how and when the frame allocator reaches `operator new` - is being addressed in P3980R0[8] and by the author's own contributions. Allocator propagation - how the allocator flows to child operations through the environment - is an engineering problem with known solutions. Categorization of compound I/O results into the three channels (`set_value` , `set_error` , `set_stopped`) is the subject of P4054R0[10] and P4053R0[9], not this paper.

---

3. The Claim

A standard task type provides a lingua franca. It eliminates pairwise bridges. It gives the ecosystem a common type that every library can accept and return. P3552R3[1] Section 3 states: "The `task` coroutine provided by the standard library may not always fit user's needs." SG1 discussion notes on P1056R1[35] record: "There can be more than one `task` type for different needs."

The claim is that `std::execution::task` serves as that lingua franca. This paper stress-tests that claim.

---

4. `destructible`

P2300R10[3] defines `queryable` in [exec.queryable.concept][24] as follows:

```
template<class T>
  concept queryable = destructible<T>;
```

We will return to this.

---

5. Two Libraries, Two Environments

The `Environment` parameter is an extension point. Two libraries will define two environments. The progression below begins with empty environments and escalates through standard type mismatches, custom queries, and NVIDIA's deployed GPU environment. Each step is individually reasonable. No step is recoverable.

5.1 From trivial to standard divergence

Library A	Library B
`struct env_a {};`	`struct env_b {};`
`task<int, env_a> compute();`	`task<int, env_b> fetch();`

Both environments are empty. Both produce identical default behavior. The types are incompatible - a function accepting `task<int, env_a>` cannot accept `task<int, env_b>`. A converting constructor (`env_a(auto const&)`) compiles but discards everything; the moment either environment carries state, the constructor must know what to extract.

The problem deepens with standard nested types. If Library A binds a concrete scheduler for performance - exactly what Asio users do when they replace `any_io_executor` with `io_context::executor_type` - the types diverge further:

```
struct env_a {
    using scheduler_type = my_thread_pool;
};
```

Library B uses the default type-erased `task_scheduler`. The `scheduler_type` mismatch means the inner task cannot extract the scheduler it needs. Add a second axis - `allocator_type` - and the construction protocol cannot reconcile the choices. Still just standard nested types. No custom queries yet. Already incompatible.

5.2 Custom queries and `write_env`

Library A needs to propagate tenant context in a multi-tenant service:

```
struct get_tenant_id_t
    : forwarding_query_t {};
inline constexpr get_tenant_id_t
    get_tenant_id{};

struct env_a {
    using scheduler_type = my_thread_pool;
    tenant_id tid;
    auto query(get_tenant_id_t) const
        -> tenant_id { return tid; }
};
```

One custom query. Completely reasonable. Library B knows nothing about `get_tenant_id`. P2300R10[3] provides `write_env` in [exec.write.env] - the caller names each missing query and provides its value by hand:

```
auto adapted = write_env(
    compute(),
    prop{get_tenant_id, my_tid});
```

It compiles. It works. But the moment Library B also defines a custom query - `get_connection_pool` - the caller must inject `get_tenant_id` into one side ***and*** `get_connection_pool` into the other. The caller must know every custom query from every library, by name, and inject them all manually. There is no discovery mechanism - no way to ask an environment "what queries do you need?" For two libraries with one custom query each, a determined caller can make it work. Tedious, but possible.

5.3 What do real domains look like?

NVIDIA's reference implementation defines custom forwarding queries in nvexec/stream/common.cuh[36]:

```
struct get_stream_provider_t
    : __query<get_stream_provider_t>
{
    static constexpr auto
    query(forwarding_query_t) noexcept
        -> bool { return true; }
};
```

The `stream_provider` carries a CUDA stream, pinned and managed memory resources, a stream pool, a task hub, and a priority level:

```
struct stream_provider
{
    cudaError_t status_{cudaSuccess};
    std::optional<cudaStream_t> own_stream_{};
    context context_;
    std::mutex custodian_;
    std::vector<std::function<void()>>
        cemetery_;
};
```

Where `context` carries:

```
struct context
{
    std::pmr::memory_resource*
        pinned_resource_;
    std::pmr::memory_resource*
        managed_resource_;
    stream_pools_t* stream_pools_;
    queue::task_hub* hub_;
    stream_priority priority_;
};
```

This is injected into the environment via `make_stream_env`:

```
auto make_stream_env(
    BaseEnv&& base_env,
    stream_provider* sp) noexcept
{
    return __env::__join(
        prop{get_stream_provider, sp},
        static_cast<BaseEnv&&>(base_env));
}
```

This is not hypothetical. This is deployed code in the `std::execution` reference implementation. Now imagine a networking domain with its own custom queries:

GPU domain (nvexec)	Network domain
`get_stream_provider` -> `stream_provider*`	`get_ssl_context` -> `ssl::context*`

GPU domain (nvexec)	Network domain
`get_stream` -> `cudaStream_t`	`get_connection_pool` -> `pool*`
pinned/managed memory resources, stream pool, ...	...

Neither environment can construct itself from the other. The caller would need to inject `get_stream_provider` - which requires a `stream_provider*` pointing to a live object managing CUDA resources - plus `get_ssl_context`, `get_connection_pool`, and every other custom query from every domain involved. The caller must construct the domain-specific objects correctly, manage their lifetimes, and know the semantic requirements of each query. `write_env` does not help with any of this. It is a syntactic mechanism for injecting key-value pairs. The semantic burden is entirely on the caller.

5.4 What about the standard queries?

The standard defines eight forwarding queries: `get_scheduler`, `get_allocator`, `get_stop_token`, `get_domain`, `get_delegation_scheduler`, `get_forward_progress_guarantee`, `get_completion_scheduler<Tag>`, and `get_await_completion_adaptor`.

A conversion layer could forward these. But NVIDIA already defines custom queries (`get_stream_provider`, `get_stream`) that are not in this list. Any domain that needs custom queries - GPU, networking, database, audio - is outside the standard set. The standard queries are the ***minimum***. The `Environment` parameter exists precisely so domains can add ***more***. P3552R3[1] itself demonstrates a custom `get_value_t` query in its own Section 4.7.

The design ***encourages*** custom queries. A conversion that handles only standard queries defeats the purpose of the open protocol.

5.5 `destructible`

No general conversion exists. The query set is open by design. `write_env` requires foreknowledge of every query. Forwarding standard queries misses the custom queries that justify the Environment's existence.

Good stewardship of the standard means shipping features narrow and widening with evidence. The committee has practiced this consistently - `std::string_view`, `std::span`, the Lakos Rule.

Every concept in the sender/receiver model eventually bottoms out at `queryable`:

Concept	Refines
`scheduler`	`queryable`, `copyable`, `equality_comparable`, ...
`sender`	`queryable`, `move_constructible`, ...
`receiver`	`queryable`, `move_constructible`, ...
`queryable`	

[exec]	[exec.queryable.concept]
6,607 lines	2 lines

`queryable` is `destructible`. It cannot be narrowed after C++26 ships.

N libraries with N different environments produce N incompatible task types with no general conversion path.

Jonathan Müller wrote in P3801R0[6]:

> *"As a standardization committee, we are drafting a standard that should outlive us. We are not working on some open-source library, we are designing the foundation for an entire ecosystem."*

5.6 If `Environment` has a default, who customizes it?

Nobody	Somebody
The parameter is unnecessary.	The lingua franca breaks.

6. The Asio Precedent

The only production two-parameter coroutine type is Asio's `awaitable<T, Executor>`. Asio provides a type-erased default - `any_io_executor` - that works for the majority of users. Most users stay on the default and succeed. But when users deviate - for performance, for concrete executor binding - the type incompatibility surfaces. Five independent reports illustrate the kind of failure the `Environment` parameter will produce at greater scale:

- A user cannot `co_await` a custom awaitable inside an Asio coroutine (StackOverflow #69280674[16], 11 upvotes, 3,014 views): "I am struggling to find out how to implement custom awaitable functions."
- A user describes Asio's executor behavior in coroutines as "rather confusing" and "very irritating" (StackOverflow #78593944[17], 990 views).
- A user hits compilation errors from a concrete executor type (StackOverflow #79115751[18]).
- A user cannot `co_await` across `io_context` boundaries (StackOverflow #73517163[19]).
- A user cannot create custom awaitable functions (GitHub asio #795[20]).

Asio's case is mild. The executor concept has a closed interface. `any_io_executor` provides an escape hatch at the cost of one virtual dispatch per operation. Users who need performance can opt into a concrete executor and accept the type incompatibility.

P3552R3[1]'s `Environment` has an open interface. The query set is unbounded by design - no fixed set of operations to erase against. The escape hatch that saved Asio cannot exist here. The mild case already exhibits the predicted symptoms. The `Environment` is structurally riskier.

---

7. Design Intent

The C++20 coroutine mechanism separates the return type from the promise type by design. Nicol Bolas explained the rationale on StackOverflow[14]:

> *"That a function is a coroutine is intended to be an **implementation detail** of the function. Nothing in a function declaration requires it to be a coroutine. Including the return type."*

The separation was designed so that a library can change its internals from callbacks to coroutines without changing the return type. `task<T, Environment>` puts sender policy in the return type, violating this separation.

Bolas continued[14]:

> *"Separating the two is intended to allow existing types that support continuations to be re-implemented internally as coroutines."*

When the Environment changes, the return type changes, and every caller breaks. The separation was designed to prevent exactly this.

Gor Nishanov formalized the consequence in P1362R0[4] Section 3.7:

> *"The separation is based on the observation that coroutine types and awaitables could be developed independently of each other by different library vendors. In a product, one could use N coroutine types that describe how coroutine behaves and M awaitables describing how a particular asynchronous API should be consumed. Users can freely mix and match coroutines and awaitables as needed."*

Nishanov quantified the cost of violating this separation in the same section:

> *"[A design that] does not separate their customization points into separate categories and every coroutine_suspend/coroutine_resume depends both on the promise and the awaitable...would require providing implementation for N * M customization points, whereas Coroutine TS will get by with just N + M."*

The `Environment` parameter reintroduces this coupling. K libraries with K different environments require K***(K-1)/2 pairwise*** `write_env` ***adaptations, each requiring foreknowledge of every custom query on the other side. The Environment ties the task type to the execution context, collapsing the promise/awaitable separation back into the N***M cost Nishanov warned against.

Nishanov reinforced the principle in P0975R0[5]:

> *"Unlike most other languages that support coroutines, C++ coroutines are open and not tied to any particular runtime or generator type and allow libraries to imbue coroutines with meaning."*

And at CppCon 2017[22]:

> *"Different category of people have different desires for their asynchronous runtime so coroutines are open ended, they can hook up to anything you want."*

The coroutine mechanism was designed to prevent the problem that Section 5 documents. The `Environment` reintroduces it.

---

8. The Ecosystem

Nine coroutine libraries are surveyed below. Asio is the most widely deployed C++ async library; the standard proposal carries normative weight. The convergence argument is not about counting libraries - it is about independent design decisions. Seven independent teams, solving different problems, all arrived at one parameter. The two that added a second parameter both needed an escape hatch: Asio provides `any_io_executor` ; P3552R3[1] does not.

Library	Declaration	Params	Source
asyncpp	`template<class T> class task`	1	task.hpp
Boost.Cobalt	`template<class T> class task`	1	task.hpp
Capy	`template<class T = void> struct task`	1	task.hpp
cppcoro	`template<typename T> class task`	1	task.hpp
aiopp	`template<typename Result> class Task`	1	task.hpp
libcoro	`template<typename return_type> class task`	1	task.hpp
folly::coro	`template<typename T> class Task`	1	Task.h
Boost.Asio	`template<class T, class Executor = any_io_executor> class awaitable`	2	awaitable.hpp

Library	Declaration	Params	Source
P3552R3 (std)	`template<class T, class Environment> class task`	2	task.hpp

The interface converged. The machinery diverged:

Library	Invariant enforced through promise
folly::coro	Cancellation token, fiber scheduler, async stack traces
Boost.Cobalt	Intrusive list cancellation, Asio executor binding
Google `Co`	Immovable prvalue-only `co_await` - prevents dangling references
Capy	`io_env` propagation: executor, stop token, frame allocator
cppcoro	Symmetric transfer, lazy start
P3552R3 `task`	`affine_on` scheduler, sender environment, stop token via connect

Jonathan Müller describes Google's approach in P3801R0[6]:

> *"Google's coroutine library has a pure library solution: Their default coroutine type `Co` is immovable and `co_await` takes it by-value. That way you can only `co_await` prvalue coroutines, which makes it impossible to have dangling references."*

Google needed a safety property that no other library provides. One template parameter. The promise enforces the invariant. Callers see `Co<T>`. The one-parameter design let them build it without fragmenting the ecosystem.

Domain-specific task types interoperate through the C++20 awaitable protocol. The cross_await[25] repository (Klemens Morgenstern) contains four cross-library composition examples, 51-105 lines each. Sender bridges follow the same pattern (P4055R0[11], P4056R0[12]). The ecosystem independently arrived at the design that avoids the problem documented in Section 5.

---

9. Concepts Mitigate the Risk

The fix is a general pattern: define a concept that constrains awaitables, not task types. The promise remains the extension point. The return type stays clean.

`IoAwaitable` (P4003R0[2]) is one realization:

```
template<typename A>
concept IoAwaitable =
    requires(A a, std::coroutine_handle<> h,
             io_env const* env)
    {
        a.await_suspend(h, env);
    };
```

Any type satisfying the concept works. The task type remains `task<T>` - one parameter. The promise delivers the domain environment through `await_transform`. New domains create new promises, not new template parameters.

The concept constrains the awaitable, not the task type. Any task type whose promise propagates `io_env` can `co_await` any `IoAwaitable` - N task types plus M awaitables equals N+M implementations. The N+M property holds because the concept does not name the task type.

Bolas explained the role of `await_transform` [15]:

> *"To declare the existence of such a function is to declare that your promise **cannot work** unless it instruments the awaitable... It is also useful for being able to actively shut off specific awaitables, or to only **allow** the use of specific awaitables."*

A concept with a domain-specific `await_suspend` signature gets compile-time rejection of foreign awaitables for free. `IoAwaitable` is one such concept. It is not the only one. The pattern generalizes to any domain.

`IoAwaitable` is a concept, not a type. It constrains what a promise can `co_await`, not what a coroutine returns. A promise that does not define `await_transform` for `IoAwaitable` gets a compile-time error - not a silent type mismatch. The return type stays `task<T>` regardless. This is the difference between concept-level and type-level fragmentation: `task<T, Environment>` changes the return type when the Environment changes, breaking every caller. A concept leaves the return type alone and lets the promise declare which domains it supports.

---

10. Frequently Raised Concerns

Q1: Nicol Bolas is not normative. Bolas is explaining the rationale, not writing the standard. The normative backing is Nishanov's P0975R0[5] ("open and not tied to any particular runtime") and P1362R0[4] (N+M argument), both published WG21 papers. Even setting aside design-intent sources, the empirical evidence in Section 8 - seven independent libraries converging on one-parameter designs - demonstrates that the ecosystem treats the principle as operative.

Q2: `IoAwaitable` is the author's own design. `IoAwaitable` is one realization of the principle. The cross_await bridges (Section 8, Klemens Morgenstern, independent author) demonstrate the same principle without `IoAwaitable`. Google's `Co<T>` demonstrates it without `IoAwaitable`. The principle is: domain-specific invariants belong in the promise, not in the return type.

Q3: The Environment parameter serves a real need. Section 2.1 grants this. The paper does not argue that the Environment is useless. It argues that the Environment creates a structural risk to task type diversity. The question is whether the risk is worth accepting for domains outside the sender model.

Q4: Production libraries do not interoperate anyway. The cross_await bridges in Section 8 refute this. Four working examples, 51-105 lines each. The C++20 awaitable protocol is the interop surface. Diversity with interoperability is the design intent (Nishanov P1362R0[4]: "mix and match").

Q5: A default Environment will fix the fragmentation. Section 2.1 already grants the default. Even with a default, the parameter exists so users provide non-default environments. The moment they do, fragmentation begins. Adding a default delays the risk; it does not prevent it. Section 5 traces the progression from empty environments through NVIDIA's deployed queries - the default does not change the outcome.

Q6: These issues will be fixed in a future revision. Section 2.1 already separates engineering gaps (off-limits) from structural findings. The open query set and the absence of a general conversion mechanism are consequences of the design decision to make the Environment an open protocol. They cannot be fixed without closing the protocol - which would break every custom query, including NVIDIA's (Section 5.3).

Q7: A standard task type provides a lingua franca. The benefit is real. The question is whether `task<T, Environment>` delivers it. Section 5 shows that the lingua franca holds only when everyone uses the same Environment. The moment two libraries use different environments, the lingua franca breaks - and the parameter exists precisely so that users provide non-default environments. K libraries with K environments require K***(K-1)/2 pairwise adaptations (Section 7) - the N***M cost Nishanov warned against. A concept is a stronger lingua franca than a concrete type with a second template parameter that fragments on contact with itself.

---

11. Conclusion

The `Environment` parameter in P3552R3[1] creates a structural risk to the task type diversity that C++20 coroutines were designed to enable. The risk is documented by the specification (the open query set, Section 4), by the reference implementation (NVIDIA's custom queries, Section 5.3), by the only production precedent (Asio, Section 6), and by `task`'s own author (Section 5.5).

Acknowledgments

The author thanks Gor Nishanov for the coroutine model's explicit support for task type diversity; Peter Dimov for identifying the frame allocator propagation gap; Klemens Morgenstern for Boost.Cobalt and the cross_await bridges; Dietmar Kühl for P3552R3[1], P3796R1[7], and `beman::execution`; Jonathan Müller for confirming the symmetric transfer gap in P3801R0[6] and for documenting Google's `Co` design; Aaron Jacobs for the CppNow 2024 presentation on coroutines at scale[23]; Steve Gerbino for the constructed comparison and bridge implementations; and Mungo Gill, Mohammad Nejati, and Michael Vandeberg for feedback.

References

WG21 Papers

1. P3552R3 - "Add a Coroutine Task Type" (Dietmar Kühl, Maikel Nadolski, 2025). https://wg21.link/p3552r3
2. P4003R0 - "Coroutines for I/O" (Vinnie Falco, Steve Gerbino, Mungo Gill, 2026). https://wg21.link/p4003r0
3. P2300R10 - "std::execution" (Michał Dominiak et al., 2024). https://wg21.link/p2300r10
4. P1362R0 - "Incremental Approach: Coroutine TS + Core Coroutines" (Gor Nishanov, 2018). https://wg21.link/p1362r0
5. P0975R0 - "Impact of coroutines on current and upcoming library facilities" (Gor Nishanov, 2018). https://wg21.link/p0975r0
6. P3801R0 - "Concerns about the design of `std::execution::task`" (Jonathan Müller, 2025). https://wg21.link/p3801r0
7. P3796R1 - "Coroutine Task Issues" (Dietmar Kühl, 2025). https://wg21.link/p3796r1
8. P3980R0 - "Task's Allocator Use" (Dietmar Kühl, 2026). https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2026/p3980r0.html
9. P4053R0 - "Sender I/O: A Constructed Comparison" (Vinnie Falco, Steve Gerbino, 2026). https://wg21.link/p4053r0
10. P4054R0 - "Two Error Models" (Vinnie Falco, 2026). https://wg21.link/p4054r0
11. P4055R0 - "Consuming Senders from Coroutine-Native Code" (Vinnie Falco, Steve Gerbino, 2026). https://wg21.link/p4055r0
12. P4056R0 - "Producing Senders from Coroutine-Native Code" (Vinnie Falco, Steve Gerbino, 2026). https://wg21.link/p4056r0
13. P4058R0 - "The Case for Coroutines" (Vinnie Falco, 2026). https://wg21.link/p4058r0

StackOverflow and GitHub

14. Nicol Bolas, "Why is the promise type separated from the coroutine object?" StackOverflow #68167497. https://stackoverflow.com/questions/68167497
15. Nicol Bolas, "Why is `promise_type::await_transform` greedy?" StackOverflow #76110225. https://stackoverflow.com/questions/76110225
16. "co_await custom awaiter in boost asio coroutine" StackOverflow #69280674. https://stackoverflow.com/questions/69280674
17. "Boost Asio: Executors in C++20 coroutines" StackOverflow #78593944. https://stackoverflow.com/questions/78593944
18. "Boost asio using concrete executor type with c++20 coroutines causes compilation errors" StackOverflow #79115751. https://stackoverflow.com/questions/79115751
19. "Can I co_await an operation executed by one io_context in a coroutine executed by another in Asio?" StackOverflow #73517163. https://stackoverflow.com/questions/73517163
20. "How to create custom awaitable functions" GitHub asio #795. https://github.com/chriskohlhoff/asio/issues/795

21. "Why can not `start_detached` an `exec::task`?" stdexec #1594. https://github.com/NVIDIA/stdexec/issues/1594

Talks

22. Gor Nishanov, "Naked coroutines live (with networking)," CppCon 2017. https://www.youtube.com/watch?v=UL3TtTgt3oU
23. Aaron Jacobs, "C++ Coroutines at Scale - Implementation Choices at Google," CppNow 2024. https://www.youtube.com/watch?v=k-A12dpMYHo

Other

24. C++ Working Draft (Richard Smith, ed.). https://eel.is/c++draft/
25. Klemens Morgenstern, cross_await - "co_await one coroutine library from another" (2026). https://github.com/klemens-morgenstern/cross_await
26. cppcoro - C++ coroutine library (Lewis Baker). https://github.com/lewissbaker/cppcoro
27. libcoro - C++20 coroutine library (Josh Baldwin). https://github.com/jbaldwin/libcoro
28. asyncpp - Async coroutine library (Péter Kardos). https://github.com/petiaccja/asyncpp
29. aiopp - Async I/O library (Joel Schumacher). https://github.com/pfirsich/aiopp
30. bemanproject/task - P3552R3 reference implementation. https://github.com/bemanproject/task
31. folly::coro - Facebook coroutine task type. https://github.com/facebook/folly/blob/main/folly/experimental/coro/Task.h
32. Boost.Cobalt - Boost coroutine task type (Klemens Morgenstern). https://github.com/boostorg/cobalt/blob/develop/include/boost/cobalt/task.hpp
33. Capy - Coroutine primitives library (Vinnie Falco). https://github.com/cppalliance/capy
34. Corosio - Coroutine-native networking library. https://github.com/cppalliance/corosio
35. P1056R1 - "Add lazy coroutine (coroutine task) type" (Lewis Baker, Gor Nishanov, 2019). https://wg21.link/p1056r1
36. NVIDIA, nvexec/stream/common.cuh - GPU stream environment queries. https://github.com/NVIDIA/stdexec/blob/main/include/nvexec/stream/common.cuh